GRAIN

How everything works

A plain-language guide to every decision, every piece of architecture, every table, every function and every number in the Grain app and its food engine. Written for the founder, so nothing in the product is a black box.

Built from the code in the `deficit` repository (app) and the `grain_engine` repository (engine lab), plus both build logs. State of the code as of 12 November 2026. 333 automated tests pass.

Contents

1. How to read this, and the words you will meet
2. The story in one page
3. The big picture: who talks to whom
4. A tour of the repository
5. The app shell: screens, gates and the installable web app
6. Where data lives: the key/value idea
7. Sign in and accounts
8. The database, table by table
9. The food engine, stage by stage (the "RAG" part)
10. The three food tables and where their numbers come from
11. Voice notes
12. GrainAI, the roast
13. The maths: targets, budget, adaptive maintenance, predictions
14. Honest history: full days, breaks, fresh starts
15. Training: logger, set maths, records, offline sync
16. Describe a workout and import history (no AI)
17. Progress, and the one line only Grain can write
18. Programs, meal plans and coaches
19. Consent, privacy, export and delete
20. The back office: access, premium, CRM, stats
21. Hosting and deployment
22. Free tiers: how much Grain can handle today
23. What it costs when it grows
24. Known weaknesses and the engine roadmap
25. The decision log, dated
26. Appendix A: every storage key
27. Appendix B: every engine action and environment variable
28. Appendix C: every file in `src/lib` and what it owns

1. How to read this, and the words you will meet

Each chapter stands alone. If you only want the engine, go to chapter 9. If you want to know when the free tiers run out, go to chapter 22. Technical words are explained the first time with an everyday comparison, and the most common ones are collected here so you can come back to them.

Word	What it means in Grain	Everyday picture
App / PWA	Grain is a website that behaves like an app. "PWA" means progressive web app: it can be added to the home screen and opens full screen with no browser bar. There is no App Store build yet.	*A shop that rents a stall in the mall (the browser) but puts up its own sign and door so it feels like its own store.*
Expo / React Native	The toolkit the app is written in. It is designed for phone apps but is exported here as a website. The same code could later become a real iOS or Android app.	*Writing a recipe once in a way that works on a gas stove today and an induction hob tomorrow.*
Vercel	The company that hosts the website files and hands them to visitors. It rebuilds the site every time code is pushed to GitHub.	*The printer and courier: every time you save a new version of a leaflet, they print it and put it in every letterbox.*
Supabase	The backend provider. It gives Grain a Postgres database, sign in (Auth), and a place to run server code (Edge Functions).	*A rented office building with a vault (database), a reception desk that checks IDs (Auth) and a back room where staff do work customers cannot see (Edge Functions).*
Postgres	The database: tables with rows and columns, like spreadsheets that enforce rules.	*A filing cabinet where every drawer has a strict form.*
Row-level security (RLS)	A rule inside the database that says "a signed in person can only see rows that belong to them". It is enforced by the vault itself, not by the app, so a bug in the app cannot leak another person's meals.	*Lockers in a gym: your key only opens your locker, no matter what you try.*
Edge Function	A small program (`food`) that runs on Supabase's servers, written for the Deno runtime. The food engine lives here because it holds secret keys that must never be in the browser.	*The kitchen. Customers (the app) send an order ticket; the kitchen uses ingredients (API keys) the customers never touch.*
LLM / model	A large language model such as gpt-oss-120b on Groq. Grain uses it to read a meal description and to judge matches, never to invent the final numbers.	*A very well-read assistant with a bad memory for exact figures. Great at understanding, unreliable at arithmetic.*
Groq	A company that runs open models very fast and cheaply. Grain's meals and voice notes go there.	*A fast food counter for AI: limited menu, served in seconds.*

Token	A chunk of text, roughly three quarters of a word. Model providers charge and rate-limit by tokens. A meal costs about 3,000 tokens end to end.	*Mobile data: you pay per megabyte, not per message.*
API / API key	A door another service opens for programs, and the password for that door. Grain holds keys for Groq and USDA.	*A staff pass for a building you do not own.*
JSON	The text format everything is stored and sent in: `{"kcal": 120, "proteinG": 3}`.	*A labelled list, written in a way both humans and programs can read.*
Key/value store	Storage where each piece of data has a name (key) and content (value). Grain stores almost everything this way.	*Coat check: hand over a coat, get a numbered ticket, use the ticket to get the coat back.*
Cache	A saved copy of an answer so the same question does not have to be worked out twice. Grain caches engine answers for seven days.	*Writing the answer in the margin of the textbook so you do not redo the sum.*
CORS / allowed origins	A browser rule: a website may only call servers that explicitly allow that website. The engine only answers calls from `app.getgrain.fit` and localhost.	*A guest list at the door.*
Anon key vs service role key	Supabase gives two keys. The anon key is public and is in the app; RLS limits what it can do. The service role key bypasses RLS and lives only inside the Edge Function.	*A visitor badge versus the building manager's master key.*
Deterministic	Same input, same output, every time. All of Grain's arithmetic is deterministic; only the model's understanding of text is not.	*A calculator, as opposed to asking a friend.*
RAG	"Retrieval augmented generation": look up real documents first, then let the model answer using them. Grain's engine is a retrieval-grounded pipeline of this family, but it retrieves food rows by name matching, not by vector search. Chapter 9 explains the difference.	*An open-book exam instead of a closed-book one.*

2. The story in one page

You had been training for seven or eight months, found muscle gain easy and fat loss hard in a carb-rich Indian household, and started logging dinner into a ChatGPT project each night to get calories, macros and the deficit. It sometimes hallucinated. The idea was a simple app where a meal typed in plain language is worked out by a hybrid: an AI for understanding the sentence, and a real food database for the numbers, each checking the other, so the daily dashboard updates every time you eat.

That idea is still the north star, and it is now the private engine at the centre of a larger product called Grain:

- **Describe, don't scan.** A photo cannot see the ghee in the pan or the chicken under the rice. People who already know what they ate type or say it.
- **Grounded numbers.** Every food number comes from a measured source (a curated Indian dish table, India's lab-measured IFCT 2017, or USDA FoodData Central) or is visibly flagged as an estimate.
- **Arithmetic in code.** The model picks foods; code multiplies portions. A cross-check cannot catch a mistake both sides make, so weights never go through the model's maths.
- **Honest history.** Days nobody logged count as nothing, never as a deficit. A half-logged day is held out until finished.
- **An ecosystem for people who train.** Food, training with every set, programs and meal plans, imports of old history and a coach's view, all on one timeline. The engine is the moat; everything else is a service around it.
- **Audience.** Athletes, lifters, models and fitness creators, India first and now global. They weigh their food, notice a wrong number, and pay for tools that get it right.

Names: the project started as **Deficit** (you still see `deficit` in folder names and storage keys) and became **Grain**, a name about precision rather than dieting. The domain is **getgrain.fit**; the app is at **app.getgrain.fit**.

3. The big picture: who talks to whom

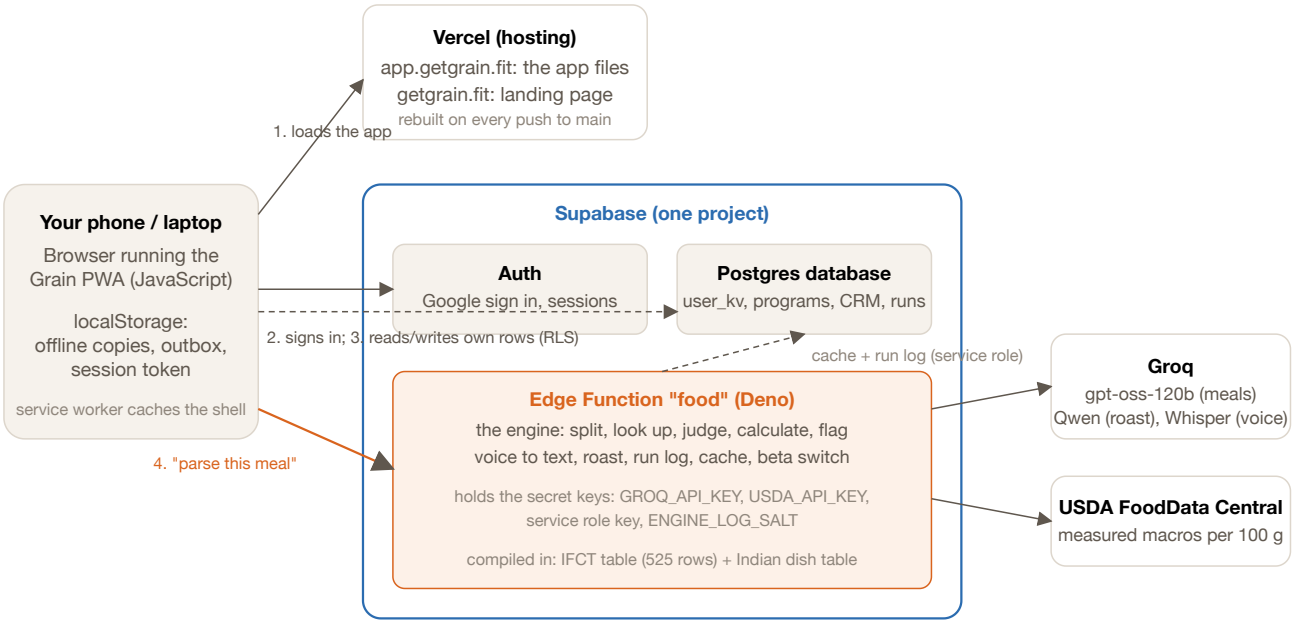


Figure 1. The whole system. Solid lines are what happens on every meal; dashed lines are database traffic. Nothing secret ever reaches the phone.

Three things to hold in your head:

1. **The app is a static website.** Vercel only hands out files. All the "thinking" happens either inside the phone's browser (the maths, the training logger, the offline parser) or inside Supabase (the engine).
2. **The phone talks to Supabase directly for data.** There is no separate Grain API server. The app reads and writes its own rows in Postgres using the public anon key, and RLS keeps everyone in their own lane.
3. **Only the Edge Function talks to Groq and USDA.** That is why the keys stay secret and why the engine can be upgraded without shipping a new app.

4. A tour of the repository

Folder or file	What lives there
`src/app/`	One file per screen, routed by file name (expo-router): `index.tsx` Overview (home), `log.tsx` Log, `train.tsx`, `workout.tsx`, `exercise.tsx`, `history.tsx`, `profile.tsx`, `program.tsx`, `programs.tsx`, `write.tsx` (write a program), `import.tsx`, `admin.tsx`. `_layout.tsx` wraps them all; `+html.tsx` is the HTML shell.
`src/components/`	Reusable pieces, grouped by feature: `dashboard/`, `log/`, `history/`, `train/`, `admin/`, `ui/`. Pure presentation; no business rules.
`src/lib/`	The brains. Every rule and every calculation, written so it can run without a screen and be unit tested. Chapters 6 to 20 are mostly about this folder. Appendix C lists every file.
`src/lib/food/`	The app side of the engine: the provider interface, the remote provider (calls the Edge Function), the local offline parser, and the curated Indian food table.
`src/lib/supabase/`	Everything that touches Supabase from the app: client, auth, the cloud key/value store, consent, programs, admin, account export and delete.
`src/lib/train/`	Training: exercise library and matching, set maths, storage with outbox, progress, programs, adherence, the workout text reader, the importer.
`supabase/schema.sql`	The whole database, as SQL you paste into the Supabase SQL editor. Safe to run again.
`supabase/functions/food/`	The engine Edge Function: `index.ts` (handler and pipeline), `portion.ts` (portion arithmetic), `ifct.ts` (IFCT search and cooked yields), `roast.ts` (GrainAI), `caches.ts` (remembered USDA answers and sign-ins), `_ifct.ts` and `_indian-foods.ts` (generated data).
`scripts/`	Data importers: `import-ifct.py` (IFCT 2017 PDF to table), `import-indb.py` (INDB spreadsheet to curated entries), `sync-indian-foods.mjs` (copies the curated table into the engine).
`landing/`	The marketing site: static HTML, CSS and JS with a working mini Grain in the hero phone. Its own Vercel project.
`public/`	`manifest.json` (PWA name and icons) and `sw.js` (service worker).
`docs/superpowers/`	Design specs and plans, including the engine v2 design.
`x.md`	The build log: every decision, dated, in plain words. The engine repo has its own.
`grain_engine/` (separate private repo)	The engine lab: the practice set (81 meals with reference answers), the scorer, results, and the engine review. The scorer imports the live engine's own code so there is one engine, not two.

Why the split between components and lib matters

Almost every rule (how a day is judged complete, how a record is detected, how a deficit becomes kilograms) is in `src/lib` with no screen attached. That is what makes 333 tests possible, and it is what lets the same rule run everywhere food is summed: the weekly card, the Home prediction, the Train progress view and the coach summary all call the same function, so they cannot disagree.

5. The app shell: screens, gates and the installable web app

5.1 What happens when the app opens

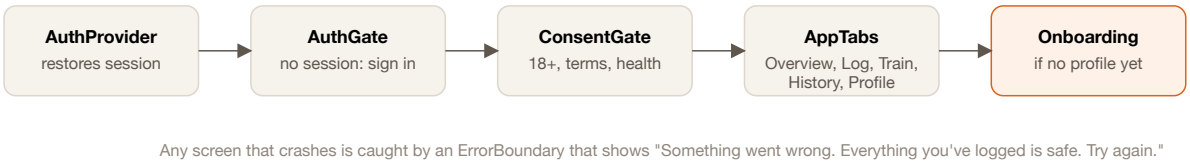


Figure 2. The gates in `_layout.tsx`. Each one is skipped when it does not apply (no Supabase configured means no sign in and no consent).

- **AuthProvider** asks Supabase whether a session exists (sessions persist in the browser's localStorage). While it asks, a spinner shows. When a session arrives it points the storage facade at the cloud (chapter 6).
- **AuthGate** shows the sign in screen when accounts are switched on and nobody is signed in. If the two `EXPO_PUBLIC_SUPABASE_*` variables are missing, the app runs entirely on the device with no account at all.
- **ConsentGate** runs once per account and per consent version. If the device already remembers that this version was agreed, it costs nothing. Otherwise it asks the database, and any doubt (offline, error) is treated as "not yet agreed", because asking again is harmless.
- **AppTabs** is the tab bar. On Overview, if there is no saved profile, the onboarding questionnaire takes over the screen.

5.2 Onboarding

Seven questions, one per screen: biological sex, age, height, weight, activity level, desired loss rate, and a review that shows the computed target live before the user has finished (the "aha" lands early). Ages under 18 are refused. Each option explains itself in a short hint ("Hard exercise 6 to 7 days a week"). The result is a `ProfileInput` saved under the profile key; nothing else is stored from onboarding.

5.3 The five tabs

Tab	What it shows	Where the numbers come from
Overview	The three hooks first: the fresh start or away card, Your week (shareable), Total deficit maintained. Then the plan (target, left to eat), the weekly forecast, the Training card, macros, weigh in, and the adaptive maintenance card.	`history.ts` gathers 21 and 45 days; `history-stats.ts`, `weekly-prediction.ts`, `adaptive-tdee.ts`, `week-report.ts`.

Log	Today's date, "Left to eat" with an arc gauge, the three macros, the meal composer (Describe, Search, Custom), plan meals when following a program, and the list of what was eaten with quantity steppers.	`log-store.ts`, `budget.ts`, `food/`, `quantity.ts`.
Train	Start a workout (empty, repeat last, or a routine), today's program day, the live logger, routines, progress, exercise pages.	`train/` (chapters 15 to 18).
History	A Food calendar and a Training calendar, then trends, the range (30, 90, Year) and charts.	`history.ts`, `history-stats.ts`, `train/progress.ts`.
Profile	Edit profile, roast switch, install button, plan line (free or premium), Download my data, Delete my account, Admin (admins only), sign out.	`access.ts`, `account.ts`, `install-prompt.ts`.

5.4 The installable app: manifest, service worker, install button

Manifest. `public/manifest.json` names the app ("Grain"), sets it to open full screen (`display: standalone`), and lists the icons. The icon URLs carry `?v=2` because installed phones kept the old logo when the files were replaced under the same name.

Service worker. A small script the browser keeps running in the background so the app can open offline. Its rules are deliberately narrow:

- Anything not on the app's own domain (Supabase auth, database, Edge Function) is never touched. It goes straight to the network.
- Build files with a content hash in their name (`/_expo/static/`, `/assets/`) are served cache first, because a changed file always gets a new name.
- Everything else, above all the HTML page that names the current bundle, is network first with the cache as a fallback. So a new deploy is picked up on the next load.

> **The bug that deleted meals**
>
> The first service worker cached every request, including database reads. The app stores a whole day as one JSON list and saves by read, modify, write. A stale cached read of today, plus a fresh write, saved the day with the newest meals missing. They were not hidden, they were gone. Lesson written into the code: offline caching and read-modify-write storage do not mix, so API traffic is never cached. The cache name was bumped to `grain-v3` to evict old copies on every phone.

Install button. Chrome offers "install this app" exactly once, early, via an event named `beforeinstallprompt`. The button lived on Profile, which mounts much later, so the offer was always missed. Fix: a tiny inline script in the HTML head catches the event before the 2.3 MB bundle even parses, stashes it on `window`, and `install-prompt.ts` hands it to the button when it appears. A prompt can be used once, so after "dismissed" the UI falls back to manual instructions.

Look. Light only, "warm instrument": bone background, warm ink, one orange accent. No dashes or minus signs anywhere in copy; the label carries the direction ("Down 1.2 kg"). Figures use tabular

numerals so columns line up.

6. Where data lives: the key/value idea

This is the most important architectural decision in the app, and the one that makes everything else simple. Grain does not have a "meals table", a "weights table" and a "workouts table" in the database. It has **one storage interface with three methods** and everything is a JSON string under a named key.

```
interface KVStore {
  get(key): Promise<string | null>;
  set(key, value): Promise<void>;
  remove(key): Promise<void>;
}
```

Picture a coat check with named hooks. The profile hangs on the hook "deficit.profile.v1". Today's meals hang on "deficit.log.v1.2026-11-12". The app never cares whether the coat room is in the phone or in the cloud.

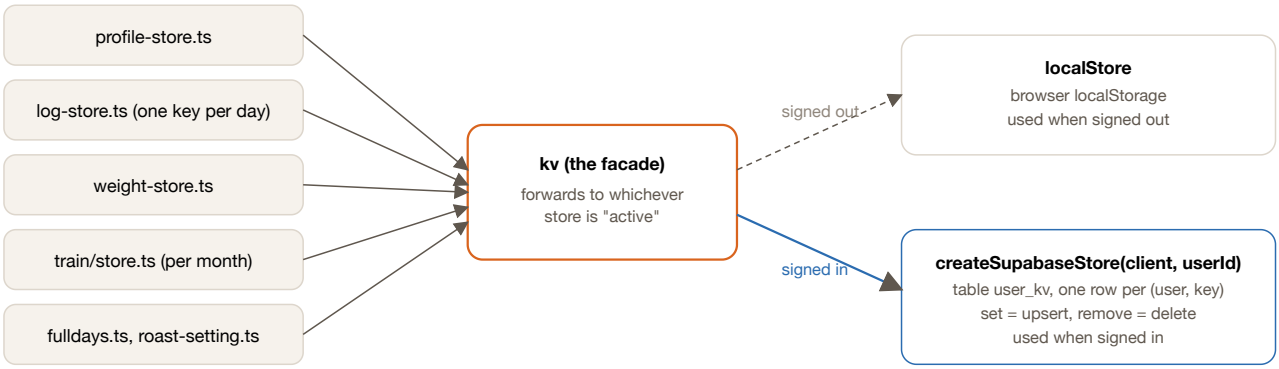


Figure 3. Every store module talks to `kv`. Sign in swaps what is behind it; no call site changes.

6.1 Sign in moves the facade, and migrates once

When a session appears, `auth.tsx` calls `setActiveStore(cloud)`. Then `migrateLocalToCloud` runs: if the cloud has no profile for this user yet, every `deficit.*` key in the phone's localStorage is pushed up. So someone who used Grain without an account keeps their data when they sign in. It runs once per device and is best effort; it never blocks sign in. On sign out the facade points back to the device.

6.2 Device-only keys are named per account

Some things must stay on the phone (offline copies, the outbox, cached consent). They are named `grain.<accountId>.<name>` via `deviceKey()`. That way two people signing in on one phone never see or sync each other's training. Deleting an account also clears that account's device keys.

6.3 The daily log, and why writes are queued

One key per calendar day, in the phone's local date (`todayKey()` uses local year, month and day, not UTC, so a meal at 11 pm in Mumbai lands on the right day). Each entry stores:

Field	Meaning
`id, label, at`	A short random id, the food's name, and when it was logged.
`kcal, proteinG, carbsG, fatG`	Totals for this entry (unit × quantity).
`serving, quantity`	The serving the unit macros describe ("1 katori (150g)" or "100g") and how many of it. Editable later.
`unitKcal, unitProteinG, unitCarbsG, unitFatG`	Per-serving macros, kept so quantity can be rescaled without going back to the engine.
`confidence, source`	The engine's trust signal at log time (high, medium, low) and where the number came from (local, usda, ifct, ai). This follows the entry into the log so a guess is never hidden.

> **The mutation queue**
>
> Add, remove and change-quantity each read the day, change it, and write it back. If two of these overlap (a multi-item meal logs several entries at once, or a double tap), the second read could see the old list and its write would drop the first entry. `log-store.ts` therefore chains every operation on the same day key behind the previous one (`serialized()`), and even reads go through the queue, because the Log screen reloads on every focus and a stale read once made a meal vanish from the list while it was still in storage.

> **7 grams became 50 grams**
>
> Quantities used to be floored at half a serving. For a food measured "per 100 g", half a serving is 50 g, so half a tablespoon of sugar (7 g, 27 kcal in the composer) was saved as 50 g and 193 kcal. Now the quantity is saved exactly as given; the stepper moves weighed foods in 5 g steps and countable foods in halves (`quantity.ts`), and "minus" at the bottom of the grid does nothing rather than jumping to a floor.

6.4 Workouts are stored per month

Finished workouts live under `deficit.workouts.v1.YYYY-MM` with an index of months under `deficit.workouts.months.v1`, so saving a workout never rewrites years of history. Chapter 15 covers the offline outbox that sits in front of this.

6.5 Trade-offs of this design, stated plainly

- **Good:** one table, one security policy, trivially testable, no schema migrations for new features, offline mode and cloud mode share all code.
- **Cost:** the database cannot query inside the JSON cheaply. Admin stats parse each day's JSON with a helper SQL function (`log_entry_count`). Reading many days is batched: the store has an optional `getMany`, and the account store answers it with one request per hundred keys, so the Overview's 45 days, a month on the calendar or a year of History each cost one or a few round trips instead of one per day.

7. Sign in and accounts

- **Google only.** The email magic link was removed on 22 October 2026: one path, nothing to deliver to an inbox, nothing to land in spam, and no SMTP to set up.
- **Implicit flow, on purpose.** Supabase supports two OAuth flows. PKCE stores a secret in the browser that started the sign in and expects the same browser to finish it; links opened in another browser or an in-app webview silently bounce back to the login screen. Implicit carries the session in the redirect itself, so it completes wherever it opens. Do not switch to PKCE without a verifier-free path.
- **Where the session lives.** In the browser's localStorage, refreshed automatically by the Supabase client.
- **Program links before sign in.** If someone opens `app.getgrain.fit/program?code=ABC123` while signed out, the code is remembered, Google sign in lands on Home, and Home opens the program.
- **Privacy copy.** The sign in screen and Profile say that meals are saved without your name to make estimates better (chapter 9.9 explains the salted hash).

8. The database, table by table

Everything below is created by `supabase/schema.sql`. "RLS on, no policies" means the table is invisible to the app's public key and only the Edge Function's master key can touch it.

8.1 Per-user data

Table	Columns	Who may touch it
`user_kv`	`user_id`, `key`, `value` (text JSON), `updated_at`. Primary key is (user_id, key).	Four RLS policies: select, insert, update, delete, each requiring `(select auth.uid()) = user_id`. Written that way, Postgres works out who is asking once per query instead of once per row, which on a 60,000 row table made a policy-only read about 14 times faster. The whole personal dataset is here.
`consents`	`user_id`, `version`, `adult`, `terms`, `health` (all must be true), `created_at`.	Users can insert and read their own; admins can read all. No update or delete policy exists, so a consent can never be edited.
`premium_access`	`user_id`, `source` ('gift' or 'subscription'), `ends_at` (null = no end), `note`, `granted_by`.	Users read their own row; admins do anything. No row means free.

8.2 Programs and coaching

Table	Columns	Who may touch it
`programs`	`owner_id`, `code` (six characters A to Z and 2 to 9, unique), `title`, `author_name`, `description`, `weeks` (1 to 52), `days` (JSON), `nutrition` (JSON meal plan or null), `exercises` (the author's custom exercises so followers have them), `published`.	Owner does anything. Anyone signed in can read a published program, or one they already follow even after it is unpublished.
`follows`	`follower_id`, `program_id`, `coach_id` (always the program's owner), `display_name`, `share_stats`, `stats` (the summary, only while sharing), `stats_at`, `started_at`. One row per (follower, program).	Followers read their own. A coach reads only rows where `share_stats` is true. Nobody can insert, update or delete directly: the functions below do it.

Functions (run with the database's authority, each checking who is asking):

- `follow_program(code, share, name)`: finds a published program by code, deletes any other follow (one program at a time), inserts or updates the follow. Sharing off drops the stored summary.

- `update_follow(id, share, name, stats)`: changes sharing or name or pushes a fresh summary; turning sharing off wipes the summary the coach could see.
- `unfollow(id)`, `my_program_followers()` (counts per program, sharing or not).

8.3 The engine's own tables

Table	Columns	Purpose
`food_cache`	`key` (engine version | model | normalised text), `value` (the parsed meal), `created_at`.	Seven-day shared cache. RLS on, no policies.
`engine_runs`	`id`, `created_at`, `user_ref` (salted hash), `arm` ('live' or 'beta'), `engine` (e.g. v5/groq/openai/gpt-oss-120b), `text`, `result`, `trace` (each stage's output, tokens and time), `latency_ms`, `cached`, `degraded`, `error`.	One row per Match. The engine's flight recorder. Indexed by date and by (user_ref, arm, date) for the daily beta cap.
`engine_corrections`	`run_id`, `user_ref`, `logged` (what the user actually logged after edits).	Every edit is a free, labelled example of the engine being wrong.
`engine_settings`	One row: `beta_enabled`, `beta_provider`, `beta_model`, `rollout_percent`, `beta_daily_cap` (default 30).	The engine switch for trying a new model on some users.
`engine_testers`	`user_id`, `note`.	People who always get the beta engine.

8.4 Admin and business

Table or function	What it is
`admins`	Who may open the admin screens. No policies: only the `is_admin()` function reads it. You add yourself once by SQL.
`app_settings`	One row: `paywall_enabled`. Everyone signed in can read it; only admins can change it. Off means everyone gets every feature.
`my_access()`	Returns, for the caller: paywall on or off, premium source and end date, whether premium is active, and whether they are an admin. The app calls this once and shows the plan line.
`admin_users(search)`	Up to 500 accounts, newest first, with joined date, last activity, days logged, meals logged and premium. Counts come from parsing `user_kv` day keys; it never returns a meal.
`admin_stats()`	Aggregates only, in India time: signups (today, week, month, by day for 30 days), people logging each day, meals per day, activation (joined in 30 days and logged at least once), second-week retention (joined 14 to 27 days ago and logged between day 7 and 13), engine runs with degraded, cached and median latency, premium counts.

`crm_creators`, `crm_metrics`, `crm_touches`	The creator CRM: a person with stage (prospect, contacted, replied, onboarded, active, partner, declined), follow-up date and contact details; every metrics snapshot per platform (followers, reach, weekly and total views, engagement) kept forever so growth shows; and every DM, call or message. Admin only.
`delete_my_account()`	Deletes the caller's row in `auth.users`; foreign keys cascade everything (kv rows, consents, programs and their follows, follows, premium). Engine logs keep only the salted reference.

31 database-level checks of these rules passed in a local Postgres before the admin shipped; the app is never trusted to keep anyone out.

9. The food engine, stage by stage

9.1 Is it RAG?

Yes in spirit, no in mechanism. Classic RAG turns documents into vectors (lists of numbers) and finds the closest ones to the question. Grain's engine instead **retrieves candidate foods by name matching** from three tables, hands those candidates to the model, and lets the model choose. Then code does the arithmetic. Call it **grounded parsing with a judge**. The reason for the simpler retrieval: food names are short, the tables are small (about 1,100 curated and IFCT rows compiled into the function, plus USDA over the network), and name matching is explainable: you can always see why a candidate appeared. The engine review in the lab notes this search is also the weakest link (it once offered "Betel leaves, big" for a Big Mac), and a proper food search with weights per household measure is on the roadmap.

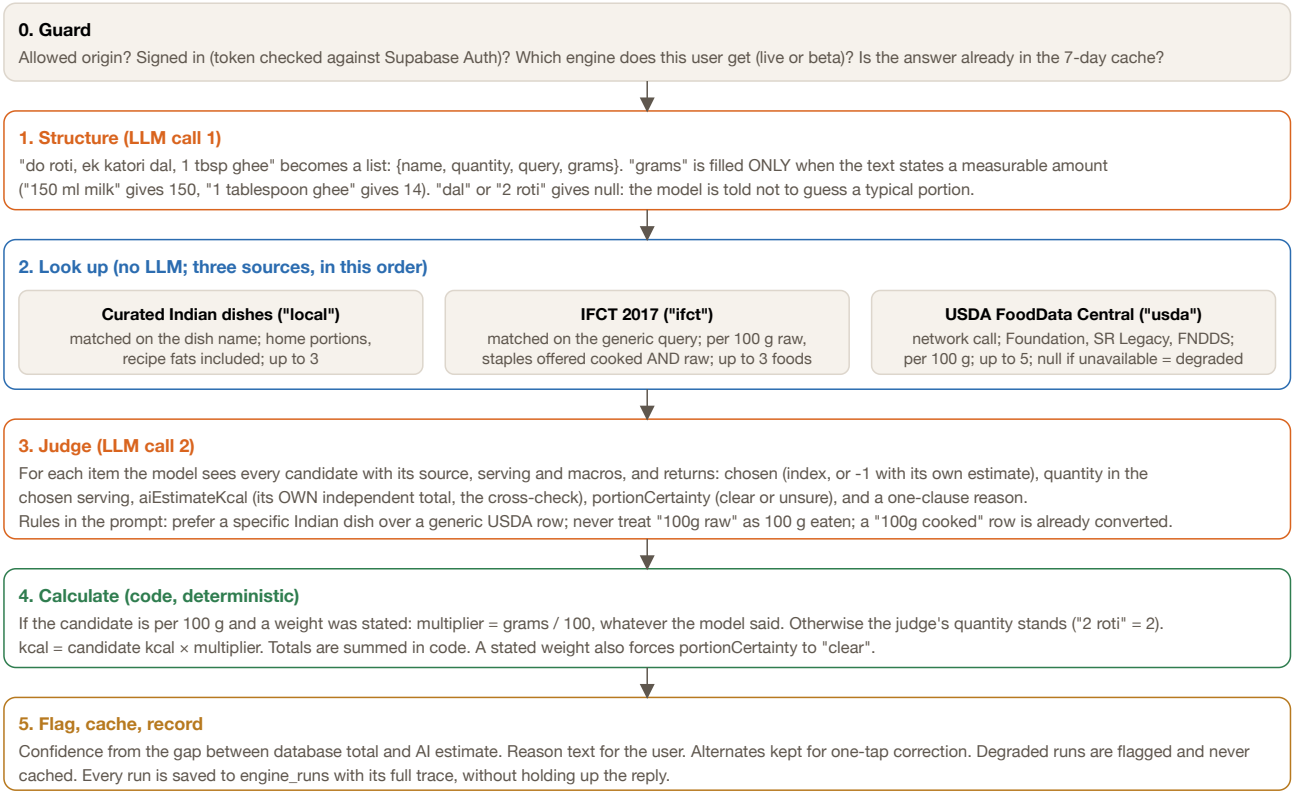


Figure 4. One "Describe a meal" request through the `food` Edge Function. Two model calls, one network lookup per item, everything else is code.

9.2 A worked example

Input: **"200g chicken breast, 1 katori dal tadka, 1 tablespoon ghee"**.

1. **Structure** returns three items: chicken breast (quantity 1, query "cooked chicken breast", grams 200), dal tadka (quantity 1, query "cooked lentils dal", grams null), ghee (quantity 1, query "ghee clarified butter", grams 14).

2. **Look up.** Chicken: the curated table has nothing exact, IFCT offers "Chicken, breast" raw at 100 g raw, USDA offers "Chicken, breast, cooked" per 100 g. Dal: the curated table offers "Dal Tadka, 1 katori (150g)", IFCT offers red gram cooked and raw. Ghee: curated "Ghee, 1 tsp (5g)" and USDA "Butter oil, anhydrous" per 100 g.
3. **Judge** picks USDA cooked chicken (the user said 200 g of something cooked), the curated Dal Tadka with quantity 1, and USDA ghee. It estimates 330, 180 and 125 kcal independently.
4. **Calculate.** Chicken: per 100 g and 200 g stated, so multiplier 2.0 (165 × 2 = 330). Dal: countable serving, quantity 1 (180). Ghee: per 100 g, 14 g stated, multiplier 0.14 (876 × 0.14 = 123). Before this rule existed, this ghee logged as 876 kcal with a "high" badge, because the model's cross-check made the same mistake.
5. **Flag.** All three gaps are small and portions are clear, so all three are "high". Total 633 kcal.

9.3 The confidence rule, exactly

```
function calcConfidence(dbTotal, aiKcal, portionCertainty, source) {
  if (!aiKcal) return 'low';            // no cross-check at all = a guess
  gap = |dbTotal − aiKcal| / max(aiKcal, dbTotal, 1);
  tol = source === 'local' ? 0.25 : 0.20; // home recipes vary more
  if (portionCertainty === 'clear' && gap <= tol) return 'high';
  if (gap <= 0.40) return 'medium';
  return 'low';
}
```

In words: **high** means the database and the model agree within 20% (25% for home dishes) and the portion was clear. **medium** means they are within 40% or the portion was guessed. **low** means they really disagree, or there was no cross-check, or no database match at all (the model's own estimate is always low). The app shows these as Matched, Likely, Unsure. The reason line for anything below high says it plainly: "Database says 180 kcal, AI estimate about 260 kcal, recipes vary."

> **An honest limitation**
>
> Confidence is the model agreeing with the database, and nobody has yet measured how often "high" is actually right. That measurement is what the practice set in the lab is for (chapter 24).

9.4 Degraded runs

A run is **degraded** when infrastructure or the judge failed: USDA was unreachable (distinct from "USDA answered and had nothing"), the model returned no decision for an item, or an item could not be placed at all. A degraded run still answers (logging must never hard-fail), shows a note ("Grounding was incomplete for this meal... try again in a minute"), and is **never cached**. An honest low-confidence result is not degraded and is cached, because it is reproducible and already flagged.

> **Why the engine once looked random**
>
> Runs made while USDA was down were being cached for a week, shared across all users. The same meal typed again returned the same wrong calories for seven days, while a phrase first parsed when USDA was healthy stayed right. That is what made the engine look "sometimes right, sometimes wildly off". Caching only fully grounded runs fixed it.

9.5 The cache

Key: `v5 | model name | lower-cased, whitespace-collapsed text`. So "200g chicken" and "200G chicken" share an answer, switching models starts a fresh namespace, and bumping `ENGINE_VERSION` (now v5) after any change to the output retires every old answer. Hits are served in well under a second and still produce a run row marked `cached`. The cache is written after the reply has gone, so it never adds to the wait.

USDA answers are remembered too (`caches.ts`). A USDA search takes about 2 seconds from India, and the same foods come up constantly ("cooked white rice", "chicken breast cooked"), even when the whole meal text is new. Each answer is kept in the instance's memory and in `food_cache` under `usda|v1|` plus a hash of the exact query, for 30 days, because USDA's data barely changes. On a meal, memory is checked first; for the rest, the stored answers and USDA are asked at the same time and a stored answer cancels its USDA search. So a known food costs nothing or one database read, and a new one waits no longer than before. "USDA unavailable" is never remembered, a search that hangs past 8 seconds counts as unavailable (a degraded run), and the answers are byte for byte the ones USDA gave.

9.6 The two prompts

Structure prompt (paraphrased): "You are a nutrition assistant for Indian food. Break this meal into items. For each return a display name, quantity as a decimal, a concise generic English search query ('cooked lentils dal', 'wheat flatbread roti'), and grams: the edible weight of the whole eaten amount, only when a measurable amount is stated. Normalise Indian portions. Return only JSON."

Judge prompt (paraphrased): "You are grounding a food log so its numbers can be trusted. For each item you get real candidates from three sources... Choose the best; prefer a specific Indian table match over a generic USDA one; if none fits use -1 and give your own estimate with a serving label. Raw vs as-eaten rules. Return quantity in the chosen serving, your own independent kcal estimate (do NOT copy the candidate), portion certainty and a short reason. Portion reference: 1 roti about 40 g, 1 katori about 150 g, 1 glass about 250 ml..."

Both calls use temperature 0 (ask for the most likely answer, not a creative one) and JSON mode, so identical meals give identical structure.

9.7 Which model, and the switch

- **Live engine:** Groq, `openai/gpt-oss-120b`, chosen after an evaluation: 16.8% median calorie error against 19.4% for llama-3.3-70b and no 400% blow-ups. A reasoning model handles the per-100 g

scaling better.

- **Pluggable:** `LLM_PROVIDER` can be `groq` or `openai`; models are set by env. Both speak the same OpenAI-compatible API, so only the URL, key and model name differ.
- **Gemini came and went:** on 28 October the engine moved to Gemini Flash through Google's compatible endpoint, and the same day went back to Groq: one key, one bill, nearly free, and Gemini is out until the practice set shows it is worth paying for.
- **Beta switch:** `engine_settings` can enable a second provider/model for testers plus a percentage of everyone else. A user's place in the rollout is fixed by their hash, so raising the percentage only adds people. A per-user daily cap (30 uncached runs) protects the budget. Everything fails towards the live engine.

9.8 Health check

The app probes `{action: 'health'}` on load. It used to report "ok" if a key string merely existed, so an expired key showed as healthy while every parse silently fell back. Now it makes a real one-token call to the provider (cached five minutes) and returns a human-readable reason (401 expired key, 429 quota, 404 retired model). The Log screen says nothing while the engine works and shows one line only when it is offline. The app asks at most once every five minutes, since every Match reports the engine's state anyway.

9.9 The run log and corrections

Every parse writes an `engine_runs` row: the text, each stage's output, token counts and milliseconds per stage, the final result, whether it was cached or degraded. When the user edits a portion or swaps a food before logging, the app fires `{action: 'correction'}` with what was actually logged, keyed to the run id. The user appears as `sha256(ENGINE_LOG_SALT + user id)`: a one-way fingerprint that cannot be turned back into a name or email, and changing the salt unlinks all old runs. The first recorded real meal ("100 gm paneer cooked with quarter tablespoon of ghee") found two bugs on day one.

9.10 Security of the function

- Deployed with `--no-verify-jwt` so the function can answer the browser's CORS preflight, but every real request is checked: the bearer token is sent to Supabase Auth and must resolve to a user. A token Auth accepted is remembered on that instance for up to five minutes (never past its own expiry), because one meal makes up to three calls with it; the trade-off is that a token revoked by signing out can still reach the engine for those few minutes. A rejected token is never remembered.
- Allowed origins: `https://app.getgrain.fit`, the old `deficit-cyan.vercel.app`, and localhost. A new domain must be added in `index.ts` and redeployed.
- Inputs are bounded: audio at most 4 MB of base64, roast text at most 300 characters and 15 items, corrections at most 30 items.

9.11 When the engine is unreachable: the offline parser

If Supabase is not configured, or the function errors, `RemoteFoodProvider` falls back to `LocalFoodProvider`: a heuristic in the phone. It splits the text on commas, "and", "+", line breaks and full stops (never a decimal point), pulls a leading number or number word ("two", "half"), strips unit words (katori, bowl, tbsp...), and ranks the curated Indian table with a simple score: exact name or alias 100, starts with 80, contains 60, token overlap 20 plus 10 per token. A stated weight is only scaled against a serving that states its weight in the same unit; otherwise it stays one serving and is flagged. The local table never claims high confidence: a strong hit is medium, a loose one is low. Type-ahead **Search** always uses this local table, because it must be instant.

9.12 Swapping a match

Each parsed item carries its other candidates. Tapping one calls `swapParsedItem`. The quantity only transfers when the new serving string is identical; otherwise it resets to 1 and confidence drops to medium with a reason, because carrying "1.5 katori" onto a "100 g raw" row once logged triple the real intake under a high badge. The replaced food stays in the list so a swap is never a dead end.

10. The three food tables and where their numbers come from

Table	What it covers	How it is built	Licence
Curated Indian dishes `local`	About 160 cooked dishes and common items with home servings: roti (1 piece, 40 g), dal tadka (1 katori, 150 g), paneer paratha, biryani, idli, chai, ghee by the teaspoon. Aliases widen matching (chapati, phulka, rotli).	`src/lib/food/indian-foods.ts`, hand curated, part derived by `import-indb.py` from the Indian Nutrient Databank with validation (Atwater check: 4P + 4C + 9F within 30% of stated kcal; range caps). Copied into the engine by `sync-indian-foods.mjs` so the two never drift.	Own curation; INDB values are facts (not copyrightable), the selection is Grain's own.
IFCT 2017 `ifct`	525 Indian ingredients measured in a lab by ICMR-NIN: millets, dals, Indian fish and meats, local vegetables. Per 100 g edible portion, raw unless published cooked (eggs).	`import-ifct.py` reads Table 1 (PDF pages 41 to 68) in layout mode, handles two column layouts, converts kJ to kcal, applies the Atwater check and range caps, and asserts the row census per food group. Dropped 3 of 528 rows. The PDF's hash is recorded.	**Open item.** The publisher requires written permission to reproduce the data in a product. Settle before marketing or investor conversations. The dataset is not named on the landing page for this reason.
USDA FoodData Central `usda`	Measured macros per 100 g for tens of thousands of foods. Foundation, SR Legacy and FNDDS (dishes as eaten). Branded foods excluded.	Live search over the network, five results, remembered for 30 days in memory and in `food_cache` (chapter 9.5). Nutrient ids 1008 (energy), 1003 protein, 1005 carbs, 1004 fat.	Public domain.

10.1 Raw versus cooked, done in code

IFCT has no cooked foods. People eat rice and dal cooked, and 100 g of cooked rice is about 33 g raw. If a raw row reached the judge unconverted, it would be read as 100 g eaten: a three-times overcount on the most-logged foods there are. So `ifct.ts` carries a short, auditable table of yields and offers staples twice:

```
cooked kcal per 100 g = raw kcal per 100 g / yield
rice (A013, A014, A015): yield 3.0     -> 356 kcal raw becomes 119 cooked
dals (chana, urad, moong, masoor, toor, rajmah): 2.5
wheat vermicelli (A023, A024): 2.2
```

Cross-check: 119 against USDA cooked white rice at 130 and the curated row at 133. Within 10%, deliberately on the low side because the raw figure carries no cooking fat. Both readings are offered ("100g cooked" first, "100g raw (uncooked weight)" second) and the judge picks the one matching how the portion was described. Eggs are detected as already cooked from their names (boiled, omlet).

A few hand aliases map vernacular to botanical English (toor dal to Red gram; chawal, poha, suji, atta). Six rows start with "Rice" and tied, so plain rice did not make the top three until an exact alias settled it.

11. Voice notes

1. The phone records with the browser's MediaRecorder in whatever format it likes (Chrome webm, Safari mp4). Maximum 60 seconds.
2. `voice.ts` decodes it and resamples to **16 kHz mono WAV** (22.05 kHz on old Safari). Speech models work at 16 kHz internally so nothing is lost; a 30-second note stays under 1 MB; and WAV is the one format every speech model reads.
3. It is sent as base64 to `{action: 'transcribe'}`. The function forwards it to Whisper large v3 on Groq with language English and a **prompt that is a sample meal**: "do roti, ek katori dal, 200g chicken breast, 1 tablespoon ghee, paneer bhurji, besan chilla, 1 scoop whey". That steers Whisper to Roman-script Hinglish and the usual spellings, because a general speech model otherwise hears Hindi food words as English ones.
4. The text lands in the box to check before matching. Nothing is logged from voice alone.

Cost: Whisper on Groq is $0.111 per hour of audio billed at a minimum of 10 seconds, about 3 paise a note. Gemini transcription was tried first and dropped the same day, with Groq Whisper kept as the single path.

12. GrainAI, the roast

Every described meal gets one brutal line after the numbers appear. It is the app's personality, on by default, with a quiet switch on Profile. Mechanics:

1. **The verdict is computed in code first** (`verdict()`): under eating if it is late (8 pm to 4 am) and the day is under half the target; disaster if the meal is 900 kcal or more, or pushes the day over target, or is 400 kcal or more with under 12% of energy from protein; clean if 30% or more of energy is protein; otherwise sloppy.
2. **The facts are worked out in code** and written as a short note, not JSON (JSON in pulls formal language out): the meal, its kcal and protein, the time, the verdict, calories left or over, protein still needed.
3. **The model** is Qwen 3.8 27B on Groq at temperature 1, with a system prompt that describes a person ("the most savage friend in the group chat"), bans soft AI phrases ("bold", "interesting choice", "keep it up"), allows swearing, and shows ten example roasts it must not reuse. gpt-oss was tried and reads every number back like a bank statement.
4. **Three candidates come back.** `parseCandidates` strips numbering, quotes and every kind of hyphen. `pickRoast` drops anything matching the banned list, prefers lines without bot tells (fewer than three numbers, under 130 characters, no "ah" or exclamation marks), and picks at random among the survivors. If nothing survives, no roast is shown.

> **The line it never crosses**
>
> Nothing about the body, weight, looks, abs or belly. Nothing sexual. Nothing about eating disorders, starving, skipping meals, self harm, depression, illness or dying. No religion, caste, race, gender, sexuality or disability. It never guesses gender ("your ex", never "bro" or "girlfriend"). It never tells anyone to eat less than their target. The reason is the audience: athletes and models are exactly the people most at risk from food and body shame, and a tracker that mocks someone for logging a brownie teaches them to stop logging it. A request to roast the belly and the sex life was declined on this basis. The prompt says all this and a word filter on the server enforces it; against the 64 lines from testing, the filter dropped every one that crossed.

No fallback to gpt-oss when Qwen is busy: Groq rate limits are per model, and a roast must never use up the capacity meal matching needs.

13. The maths: targets, budget, adaptive maintenance, predictions

None of this touches AI. Same input, same output, every time.

13.1 From profile to daily target (`targets.ts`)

```
BMR (Mifflin-St Jeor) = 10 × weight(kg) + 6.25 × height(cm) − 5 × age  (+5 men, −161
women)
Maintenance (TDEE)    = BMR × activity factor
                        sedentary 1.2, light 1.375, moderate 1.55, active 1.725, very
active 1.9
Loss rate             = chosen kg/week, clamped between 0.25 and 1.0
Daily deficit         = rate × 7,700 / 7           (7,700 kcal is about 1 kg of fat)
Target                = maintenance − deficit
Floor                 = max(BMR, 1,200): the target never goes below it
                        (if even the floor exceeds maintenance, eat at maintenance)
Protein               = 1.8 g per kg bodyweight (or the user's manual number)
Fat                   = 0.8 g per kg
Carbs                 = whatever calories remain, never negative
```

Example: 28-year-old man, 178 cm, 85 kg, moderate, 0.5 kg a week. BMR 1,818; maintenance 2,818; deficit 550; target 2,268; protein 153 g; fat 68 g; carbs 261 g. When the floor bites, a note says "We've set a safer pace. Aggressive deficits backfire and erode muscle."

13.2 Left to eat (`budget.ts`)

`remaining = target + credited exercise − consumed`. Exercise credit has a configurable "eat back" factor because eating back 100% of a fuzzy tracker estimate stalls fat loss; today the Log screen passes 0 burned, so the budget is simply target minus eaten. The gauge fraction is consumed over target clamped to 0 to 1; "Over by" replaces "Left to eat" when negative.

13.3 Learning your real maintenance (`adaptive-tdee.ts`)

The formula above is a textbook guess. Your body is the real experiment. If you ate 2,300 a day for two weeks and lost 0.5 kg, your real maintenance must have been about 2,300 + 0.5 × 7,700 / 14 = 2,575.

```
weight trend (kg/day) = least-squares slope through the weigh-ins   (robust to water-
weight noise)
estimated TDEE        = average logged intake − slope × 7,700
recommended target    = max(floor, estimated TDEE − your chosen daily deficit)
```

It refuses to guess until there is enough data: at least 7 logged days, 3 weigh-ins spanning 7 days, and **coverage of at least 70%** of the days between the first and last weigh-in. Coverage matters because the scale keeps counting through a holiday but the intake average does not; ten unlogged days of eating out would show up as weight gained on the logged intake and read as a slow metabolism.

Confidence grows with data: low, medium (10 days, 5 weigh-ins, 10-day span), high (14 days, 8 weigh-ins, 14-day span). Today's partial intake is excluded, and unfinished days are blanked first (chapter 14).

13.4 The weekly forecast (`weekly-prediction.ts`)

The window is the seven days ending **yesterday**, because a half-eaten today looks like a huge deficit and would swing the number all morning. Average (maintenance − intake) over logged days, times 7, divided by 7,700 gives "on pace to lose 0.42 kg this week". Fewer than 5 logged days shows "building" instead of a number; zero shows "fresh". Missing one or two days are implicitly filled with the logged-day average, which is what average × 7 does.

13.5 The shareable week (`week-report.ts`, `week-card.ts`)

Deficit over the last seven full days, days logged, days protein was hit, average kcal against target, weight change. Drawn on a canvas at 1080 × 1350 (Instagram portrait) in the app's own fonts, with getgrain.fit at the bottom; the preview is the exact file shared. On phones the share sheet opens to Instagram or WhatsApp; desktops download. This is the marketing loop for an audience that already posts its progress.

14. Honest history: full days, breaks, fresh starts

This is a product rule as much as a calculation, and it runs everywhere food is summed.

A day is...	When	Effect
counted	Something logged and at least half the calorie target, or the user confirmed it was really everything (a fast, a sick day) from the calendar.	In every total and average.
unfinished	Logged, but under half the target and not confirmed. Almost always breakfast logged, the rest forgotten.	Held out until finished or confirmed. Would otherwise read as a 2,000 kcal deficit.
missing	Nothing logged.	Unknown. Adds nothing as deficit or surplus. Nothing is ever guessed to fill it.

- **Today is open**, so it never counts against a streak in the morning; the streak starts from yesterday if today is not full yet.
- **A break** is three or more days in a row without a full day, after the person has logged at least one full day (the days before someone ever logged are not a lapse).
- **Away:** while a break is running, History says so plainly: last full day, days since, and that the numbers start again from the next full day.
- **Restart:** for 14 days after coming back, History leads with a fresh start that names the break with its dates and counts from the return, with 30, 90 and Year a tap away. No cheering, no guilt.
- **Total deficit maintained** = sum over counted days of (maintenance − intake), and estimated fat = that / 7,700. Targets are not versioned per day, so the current maintenance is the reference line; a fair approximation for a journey view.

> **Why**
>
> Before this, a fortnight off looked like a fortnight of discipline, and a lone breakfast looked like a 2,000 kcal deficit. A person cannot lie to themselves, so a flattering number that is not true is worse than no number. The same `dayStatus()` function now feeds the weekly card, the Home prediction, Train progress food averages, the adaptive maintenance estimate and the summary a coach sees.

15. Training: logger, set maths, records, offline sync

15.1 The exercise library and name matching

104 exercises, each with a muscle, equipment, kind (weighted or bodyweight) and aliases, ordered roughly by how common they are so ties go to the common one. Matching is built for gym shorthand: names and queries are normalised the same way (lowercase, equipment words shortened: dumbbell to db, barbell to bb; plurals folded; filler dropped; spaces removed), then scored 0 to 100: exact 100; every query word starts a candidate word 72 minus 3 per extra word; prefix 66; substring 56; small typos by edit distance (with neighbour swaps counting as one) 40 to 50. Below 30 is no match. "rdl", "ohp", "db bench", "bss", "skullcrushers" and "pullups" all land. Anything missing becomes a custom exercise in two taps.

15.2 The set maths (`workout.ts`)

```
Estimated one rep max (Epley): e1rm = kg × (1 + reps / 30), for 1 to 15 reps (1 rep =
the lift itself)
Load of a set               = kg, or for bodyweight moves: bodyweight + added kg
Set score                   = e1rm of the load   (how records are judged)
Hard set                    = ticked, not a warmup, with reps
Volume                      = kg × reps over hard sets (warmups move none)
Record                      = a hard set whose score beats every earlier score by more
than 0.05,
                              history first, then earlier sets today; a first ever
session sets none
```

Bodyweight lifts count bodyweight plus added load, so 10 pull ups at 80 kg compare fairly with 6 pull ups carrying 10 kg.

15.3 The logger's one-tap sets

- Each exercise is a card of sets: set label (W, D, F or the working number), previous, kg, reps, tick.
- **Placeholders** are last time's matching set (warmups against warmups, working against working, in order), else the nearest set above it today, else the last set from last time. So a set done the same as last time is one tap on the tick.
- Tapping a previous set copies it in. A set's number opens its type (warmup, drop, failure) and optional RPE (6 to 10).
- Every tick starts a rest timer whose length is remembered per exercise (default 90 s); the screen stays awake and the phone buzzes at the end.
- Finishing names the session from what was trained (60% or more of hard sets in one group gives Push, Pull, Legs or Core; under 20% legs gives Upper body; else Full body) unless the user named it, offers to tick sets with numbers but no tick, lists the records, and can save the layout as a routine.

15.4 Gyms are basements: the offline design (`train/store.ts`)

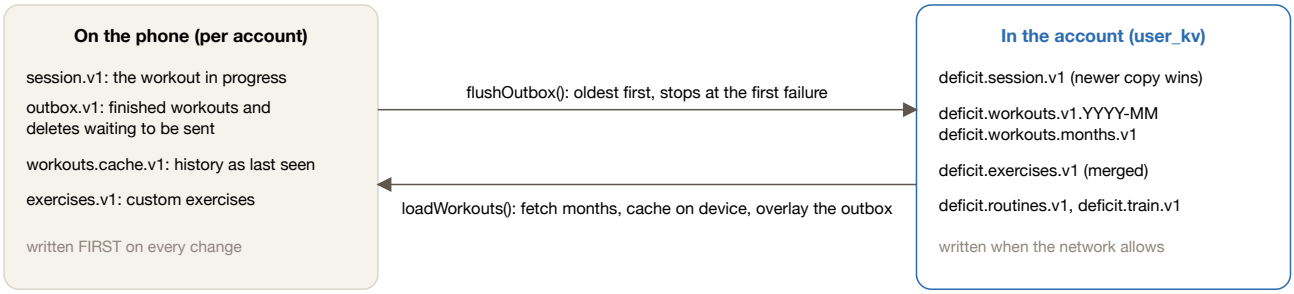


Figure 5. No set ever depends on the network. The outbox is replayed on every load and every save.

- **The workout in progress** is saved to the device on every tap and to the account when possible; on load the newer of the two copies wins. Clearing it on the device while offline writes a "cleared at" marker so a stale synced copy is ignored and removed later.
- **Finished workouts** go into an outbox as operations (put or delete). The outbox is flushed oldest first and stops at the first failure, so order is preserved. History shows outbox items immediately, overlaid on the synced list.
- **Imports** write one month at a time, index first, and need the network; running the same import twice updates rather than doubles because ids are stable.
- **Custom exercises** are merged from both copies; one made offline goes up the next time the account answers.

16. Describe a workout and import history (no AI)

`describe.ts` reads a workout typed or spoken in plain words. It is plain code on the phone: instant, free, and the same words always read the same way. Whatever it cannot place is handed back so the screen asks instead of guessing.

```
bench 80x8, 80x8, 85x6          squat 100kg 5x5          ohp 50 for 8, 8 and 7
lat pulldown 3 sets of 12 at 60  pull ups bw x 10, +10 x 6
"squats five sets of five at a hundred kilos"   (number words become digits; pounds
become kg)
```

How: normalise (× to x, "body weight" to bw, kilos to kg, pounds to lb, number words to digits, "80 and a half" to 80.5); split into pieces on lines, sentences, commas and "then"; strip leading filler ("and then I did"); move a trailing "on rows" to the front; split each piece into a name and its numbers at the first digit; carry the weight across clauses so "80x8, 8, 7" means three sets at 80; and read patterns such as weight × reps × sets, sets × reps at weight, reps for sets, and "+10" as added load on a bodyweight move. Up to 20 sets per clause. Exercise names go through the library matcher.

`importer.ts` reads a Strong export, a Hevy export, any training spreadsheet saved as CSV, or pasted notes ("Mon 3 Nov: bench 80x8, 85x6"). Names are matched to the library (Strong's "Incline Bench Press (Dumbbell)" becomes Incline dumbbell press); unknowns are listed first with a muscle guessed, to keep, rename or leave out. Rest timers and timed or distance rows are left out and counted.

17. Progress, and the one line only Grain can write

`progress.ts` compares a Monday-to-Sunday week or a calendar month with the one before. A week still running is compared with the same days last week, so a Monday is not judged against a whole week. It reports volume and hard sets, hard sets per muscle against a 10-sets-a-week mark, food averages over full days against targets, bodyweight change (against the last weigh-in before the period, else the first in it), and each lift's best estimated max with its change against the previous period (or the last session before, after a break).

Records per exercise: best estimated max, heaviest set, most reps, best session volume, and rep maxes for 1, 2, 3, 5, 8, 10 and 12 reps where a rep count only earns a line if it beats every higher count (95 × 8 already says 95 kg for 5, 3, 2 and 1).

The one line (`periodRead`) joins the three streams in priority order:

1. Bodyweight down 0.2 kg or more and median lift change up 2% or more: "Lifts up 3.0% while bodyweight fell 0.6 kg."
2. Bodyweight down and lifts within half a percent: "Strength held while bodyweight fell 0.6 kg."
3. Lifts down 2% or more, at least half the days logged, and average kcal under 85% of target: "Lifts slipped 2.4% on 1,850 kcal a day, 18% under target."
4. Protein averaging under 80% of target over 3 or more full days: "Protein averaged 110 g a day, short of your 150 g target."
5. Then: lifts up, more hard sets than before, lifts holding, or a plain count.

Only an app holding food, training and the scale together can write lines 1 to 4. That is the reason to switch from three single-purpose apps.

18. Programs, meal plans and coaches

- **Writing:** named days with exercises, sets (1 to 10) and rep ranges (1 to 50), how many weeks (1 to 52), and optionally a meal plan with daily calories, protein and named meals. Publishing gives a six-character code (no O, 0, I or 1) and a link.
- **Following:** one program at a time. Today's day sits on top of Train with one tap to start; the logger shows each exercise's rep target. **The next day is always the one after the last program day done**, so a missed session shifts the plan instead of being skipped. Week = workouts done ÷ days per week, plus one. Plan meals appear on the Log screen and log in one tap.
- **The device copy:** the followed program is cached on the phone so Train shows today's day instantly and offline.
- **Sharing with the coach** (`adherence.ts`): off until ticked, reversible, and turning it off deletes the summary. The summary covers the last 28 days ending yesterday (or since the follow started): program workouts done against planned (days per week scaled to the window), last workout date, full food days, average kcal and protein, latest weight and change. Never meals, sets or notes. It is refreshed at most every six hours. The coach's screen validates the shape of anything it receives and shows nothing rather than breaking on bad data.

19. Consent, privacy, export and delete

- **Consent** before anything else, three separate taps with nothing pre-ticked: 18 or older, terms and privacy policy, health data used for targets. Each agreement is an uneditable row with a version (`CONSENT_VERSION = '2026-11-09'`); raising the version asks everyone again.
- **Agreeing never waits on the network.** The first version blocked every new sign in when the table did not exist yet. Now the person has agreed the moment they tick, the app opens, and if the row cannot be stored it waits on the device and is sent on a later launch.
- **Download my data:** every `user_kv` row (parsed), consents, programs written, and follows, as one JSON file.
- **Delete my account:** one database function removes the auth user and everything cascades. Device keys for that account are cleared.
- **What the engine keeps about a person:** only a salted hash. Meal text is kept to improve the engine, as the sign in screen says. The landing site has a privacy policy and terms.

20. The back office: access, premium, CRM, stats

- **Access model:** one subscription for everyone, price not set. The paywall switch stays off until payments exist; off means everyone gets every feature. `hasPremium = !paywall || premiumActive`. Admins gift premium by hand with no end date or for 3, 6 or 12 months; a gift shows on that person's Profile as "Premium, a gift from Grain". Nobody can gift themselves.
- **Users:** the list with activity counts and gifts.
- **Stats:** the numbers in chapter 8.4, refreshed every minute.
- **Creators CRM:** on a laptop (1024 px or wider) a full workspace: a strip of what needs you today (follow ups due, overdue, replied with nothing planned, reply rate, roster reach), saved views (needs action, all, roster, prospects, declined), a dense table sorted by urgency (most overdue first, then due today, then replies waiting), inline stage and follow-up edits, bulk moves, CSV export, a side panel with timeline and growth, a pipeline board with drag to move, and keyboard shortcuts (J/K move, Enter open, 1 to 7 set stage, T/W follow up tomorrow or in a week, N new, X select, slash search). Growth is follower change since each platform's previous snapshot.

21. Hosting and deployment

Piece	Where	How it ships
The app	Vercel project `deficit`, root directory `/`, domain `app.getgrain.fit` (old `deficit-cyan.vercel.app` still works).	Every push to `main` runs `npx expo export --platform web` into `dist/`. Two env variables on the project: `EXPO_PUBLIC_SUPABASE_URL` and `EXPO_PUBLIC_SUPABASE_ANON_KEY`. These are baked into the bundle at build time.
The landing page	Vercel project `grain`, root directory `landing`, domains `getgrain.fit` and `www`.	No build; files served as they are. `/go` rewrites to `/` because X ignores query strings when caching link previews, so a new path is the only way to force a fresh card.
The database	Supabase project.	Paste `supabase/schema.sql` into the SQL editor. Safe to run again.
The engine	Supabase Edge Function `food`.	`supabase secrets set GROQ_API_KEY=... USDA_API_KEY=...` then `supabase functions deploy food --no-verify-jwt`. Optional: `LLM_PROVIDER`, `GROQ_MODEL`, `OPENAI_API_KEY`, `OPENAI_MODEL`, `ROAST_MODEL`, `ENGINE_LOG_SALT`.
Local development	Your laptop.	`npm install`, `npm run web`. With no `.env` the app runs fully on-device. `npm test`, `npm run typecheck`, `npm run lint`.

The engine lab (`grain_engine`) runs the scorer with Deno against the live engine's own code, with sign in faked and the cache and run log switched off, so an evaluation never pollutes production data.

22. Free tiers: how much Grain can handle today

The numbers below combine what the code and logs record with the published free-tier limits as last known. Providers change limits; treat the table as the shape of the problem and confirm current figures in each dashboard before a launch push.

Service	Free tier limit (last known)	What Grain uses per unit	Rough ceiling
Groq (LLM)	Per model, per account. gpt-oss-120b: about 8,000 tokens per minute and about 200,000 tokens per day. Qwen (roast): about 1,000 output tokens per minute.	About 3,000 tokens per uncached meal (two calls). Cached repeats are free. A roast is a third, separate call on a separate model budget.	**About 66 new meals a day across all users**, and a handful a minute. This is the binding constraint. The engine review and the 29 October log both say the paid Developer tier is required before any marketing push.
Supabase (database)	500 MB database, 50,000 monthly active users for Auth, 5 GB egress, 500,000 Edge Function invocations a month, 2 free projects, and **projects pause after about a week of inactivity**.	A day of meals is about 1 to 2 KB; a user logging all year is under 1 MB including workouts. An `engine_runs` row with its trace is 5 to 10 KB.	Personal data: hundreds of users for years. **Engine logs are the storage hog:** 50,000 runs is roughly 250 to 500 MB. Prune traces or archive them before that.
Supabase (Edge Function calls)	500,000 invocations a month.	A described meal is up to 3 calls (parse, roast, correction) plus a health probe per app open. Say 15 calls per active user per day.	About 1,100 daily active users before the call cap. Groq runs out first.
Vercel (Hobby)	100 GB bandwidth a month, generous static requests. **Hobby plans are for non-commercial use**; a paid product on Hobby is against the terms.	The app bundle is 2.3 MB on first load and cached by the service worker after that. Landing page is kilobytes.	Roughly 40,000 fresh app loads a month. Not the constraint. **Move to Pro (about $20 a month) when charging money.**
USDA FoodData Central	1,000 requests an hour per key (api.data.gov default).	One request per food it hasn't seen in 30 days; known foods are answered from memory or `food_cache`.	Far past the Groq ceiling. A USDA outage only degrades runs with foods it hasn't seen before.
Google OAuth	Free.	One redirect per sign in.	Not a constraint.

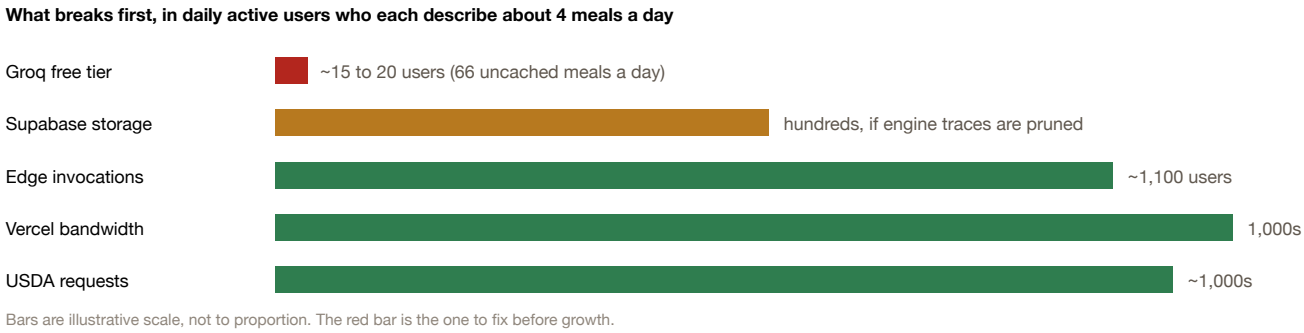


Figure 6. The ceiling today is the free Groq quota, by a wide margin. Everything else is comfortable for a first few hundred users.

22.1 Two things that are easy to forget

- **Supabase pauses idle free projects.** A week with no traffic and the database stops until someone clicks restore in the dashboard. With real users this does not happen, but a quiet test project will.
- **The Vercel Hobby plan is non-commercial.** The moment a paywall is on, the app project should be on Pro.

23. What it costs when it grows

Line	Cost	Notes from the logs
Groq, paid Developer tier, gpt-oss-120b	Under a tenth of a US cent per meal (about 3,000 tokens). A full practice-set run of 81 meals is about 6 cents.	Lifts the per-day token cap by orders of magnitude. 1,000 users × 4 meals × 30 days = 120,000 meals, well under $100 a month before caching.
Whisper on Groq	$0.111 per hour of audio, 10-second minimum: about 3 paise a note.	Negligible.
Roast (Qwen on Groq)	Similar order to a meal, one short call.	Could be made cheaper by only roasting on tap; currently every described meal.
A frontier model (if the bake-off proves it wins)	Gemini Flash was about $0.75 in and $3.75 out per million tokens, roughly a rupee per uncached meal, two after 1 January 2027.	Decision on record: pay only after the practice set shows a clear win at a known cost per meal. The cache and the daily cap keep spend predictable.
Supabase Pro	About $25 a month, 8 GB database, 100 GB egress, no pausing.	Needed when engine logs grow or users pass a few hundred.
Vercel Pro	About $20 a month per seat.	Needed for commercial use.
Domain	getgrain.fit (grain.fit was too expensive).	

Design constraint still in force from the engine v2 plan: a solo developer with an AI budget of $10 to $20 a month, and a Match that must return in under 8 to 9 seconds (it was 10 to 15 s; the first recorded meal took 6.4 s).

24. Known weaknesses and the engine roadmap

24.1 Where the engine is weak today (from the lab's review of 38 items)

- The name search is naive: "big" matched "Betel leaves, big" for a Big Mac; chicken biryani ranks ahead of chicken breast for "chicken breast".
- Cups and spoons are still turned into grams by the model's memory in stage 1. Code should do it from a weight per household measure for each food, the way FNDDS and Nutritionix do.
- A stated weight is ignored when the matched food is a home serving such as "1 katori (150g)"; only per-100 g rows get the arithmetic.
- Confidence has not been validated against truth.
- The practice set is small (27 of 30 weighed meals passing could mean 74% to 97%), and 27 reference rows still need checking against a source or a label.
- Brands are out of scope: whey, soya chunks, low-fat paneer and protein bars each have one standard row with a note that brands vary. A future "scan a label" feature is the intended answer.
- The curated table calls itself a stub in its header; its values are reasonable home-portion estimates, which is why the engine prefers it only for dishes and never lets it claim high confidence offline.

24.2 The plan that is on record

1. **Practice set**: 81 meals (53 Indian, 28 global), typed as people type them, with reference answers computed from source rows (USDA ids, IFCT codes, pack labels), never typed in. Targets: weighed meals within 5%, dishes within 15%, never off by 2× or more, lower median error than ChatGPT on calories and on protein, answer under 9 seconds.
2. **Scorer**: runs the live engine code, resumes partial runs (a full run exceeds a free day of Groq tokens), prints each failure's trace, and reports pass rates with 95% intervals and signed error.
3. **Own food database in Supabase** replacing live USDA calls: USDA as the backbone (FNDDS has thousands of dishes as eaten with real portion weights), IFCT for Indian ingredients, chains looked up from their published numbers, the user's country deciding defaults ("milk" is toned in India and 2% in the US). Open Food Facts for packaged food only once its share-alike licence is settled.
4. **Bake-off** of stronger models on the set, then pay only if one clearly wins.
5. **Ship through the beta switch**, watching corrections.

24.3 App-side things worth knowing

- Route-level code splitting (Expo's async routes) would cut the first download by another 67 KB, but on SDK 56 it leaves every screen blank. Expo calls it stable on the web from SDK 58, so it comes with an SDK upgrade. The Supabase client also carries about 90 KB of realtime and storage code Grain never uses; removing it means replacing the client and retesting Google sign-in.
- The adaptive estimate uses the current maintenance as the reference for all past days; targets are not versioned.

- No native build exists yet; on native the storage would need an AsyncStorage adapter (a comment in `kv.ts` marks the spot).
- IFCT licence, SMTP (if email ever returns), and Google Search Console verification are the open operational items.

25. The decision log, dated

Condensed from both `x.md` files. The full entries are in the repositories.

When	Decision	Why
Earlier	Describe, don't scan. Deficit becomes Grain. Audience is athletes, lifters, models, creators. Landing page with no framework.	Photos cannot see oil or weight; a name about precision; they notice wrong numbers and pay; a static page is sharp and fast.
Earlier	Arithmetic in code, not the model. Only grounded runs cached. IFCT added with cooked and raw readings.	876 kcal ghee under a high badge; a cache that froze bad numbers; a 3× overcount on rice.
21 Oct 2026	Engine v2 plan with measurable targets.	ChatGPT beat the engine on a soya-chunk meal (319 vs 733 kcal).
22 Oct	Google sign in only. Every run recorded. Beta engine switch.	Nothing to deliver to an inbox; every edit is a labelled example; building beside a live app.
23 Oct	Engine gets its own private repo. Engine first for two weeks. SEO basics on the landing page. /go path for X.	The engine is the edge and holds licensed data; accuracy is the promise; X ignores query strings.
28 Oct	Voice notes. Gemini tried and reverted the same day; Groq only. Quieter screens: no dashes, Describe first. Weekly share card. GrainAI roast, then roast on for everyone with a body-safe hard line.	One key, one bill; a calmer UI; a marketing loop; personality without shame.
29 Oct	Roast tuned on real output; Qwen stays; filter widened; no fallback to gpt-oss. Groq free tier identified as the ceiling. Practice set and scorer built. Grain goes global.	gpt-oss reads like a bank statement; limits are per model; 1,000 tokens a minute; the audience is the same person in Mumbai and Austin.
31 Oct	Scorer resumes partial runs; full engine review with 38 fixes ordered.	A full run exceeds a free day of tokens.
8 Nov	Grain becomes an ecosystem for people who train. Back office: CRM, users, stats.	The space is scattered; one timeline is the reason to switch; growth starts with athletes and creators.
9 Nov	Laptop CRM. Training joins the log. Missed days count as nothing. Consent and 18+. Import history. Programs and coaches. Nothing typed gets lost.	Admin work happens on a laptop; gyms are basements; flattering numbers lie; legal footing; bring your history; a reason for coaches to bring athletes.
10 Nov	Agreeing never waits on the network. Landing phone becomes a working mini Grain. Training on Home.	Consent blocked every sign in before its table existed.
11 Nov	Guided tour in the hero phone. Home leads with the hooks, History with calendars, professional pass.	A senior product manager's four points, all taken.

Appendix A: every storage key

Key	Holds
`deficit.profile.v1`	The profile: sex, age, height, weight, activity, rate, protein mode, created and updated times, schema version 1. Its presence in the cloud is also the "already migrated" flag.
`deficit.log.v1.YYYY-MM-DD`	One day's meals as a JSON list of entries.
`deficit.weight.v1`	All weigh-ins, one per day, sorted.
`deficit.complete.v1`	Days the user confirmed were complete despite being under half the target.
`deficit.session.v1`	The synced copy of a workout in progress.
`deficit.workouts.v1.YYYY-MM` / `deficit.workouts.months.v1`	Finished workouts per month and the index of months.
`deficit.exercises.v1` / `deficit.routines.v1` / `deficit.train.v1`	Custom exercises, saved routines, and per-exercise rest settings.
`grain.roast.v1`	Roast on or off (absent means on).
`grain.<account>.session.v1` / `outbox.v1` / `workouts.cache.v1` / `exercises.v1`	Device-only training copies, per account.
`grain.<account>.consent.v1` / `consent.pending.v1`	The agreed consent version, and one waiting to be sent.
`grain.<account>.follow.v1`	The followed program as last seen.
`grain.after_signin`	A program code opened before signing in.

Keys starting with `deficit.` are the ones migrated to the cloud on first sign in and synced thereafter. Keys starting with `grain.` stay on the device.

Appendix B: every engine action and environment variable

Action	Body	Returns
health	`{action:'health'}`	`{ok, provider, model, usda, llm}`; `llm` is a human-readable reason on failure.
parse	`{action:'parse', text}`	A ParsedMeal: items with confidence, reason and alternates, totals, an optional note, `runId`, and `cached` when served from cache.
correction	`{action:'correction', runId, logged[]}`	`{ok:true}`. Fire and forget from the app.
transcribe	`{action:'transcribe', audio (base64 WAV), mime:'audio/wav'}`	`{text}`.
roast	`{action:'roast', meal, items[], day{eatenKcal, targetKcal, proteinG, proteinTargetG, hour}}`	`{line}` or `{line:null}`.

Variable	Where	Meaning
EXPO_PUBLIC_SUPABASE_URL, EXPO_PUBLIC_SUPABASE_ANON_KEY	App (Vercel env, local .env)	Turn on accounts and sync. Public by design.
GROQ_API_KEY	Edge Function secret	Meals, roast and Whisper.
USDA_API_KEY	Edge Function secret	USDA search. Missing means every run is degraded.
LLM_PROVIDER, GROQ_MODEL, OPENAI_API_KEY, OPENAI_MODEL, ROAST_MODEL	Edge Function secret	Provider and model switches. Defaults: groq, openai/gpt-oss-120b, gpt-4o-mini, qwen/qwen3.8-27b.
ENGINE_LOG_SALT	Edge Function secret	Salt for the user hash in run logs. Changing it unlinks old runs.
SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY	Provided by Supabase to the function	Checking sign in, and the master key for cache, run log and settings.

Appendix C: every file in `src/lib` and what it owns

File	Owns
kv.ts	The KVStore interface, the local store, the facade, device-scoped keys, JSON helpers.
log-store.ts	Day keys, entries, summarising a day, the per-key mutation queue, portioned entries.
profile-store.ts, weight-store.ts, fulldays.ts, roast-setting.ts	Profile, weigh-ins, confirmed complete days, roast switch.
targets.ts, budget.ts	BMR, TDEE, deficit, floor, macros; left to eat.
adaptive-tdee.ts, weekly-prediction.ts	Learned maintenance from intake and weight slope; the week's forecast.
history.ts, history-stats.ts	Gathering day series; day status, summaries, streaks, breaks, restarts.
week-report.ts, week-card.ts	The week's numbers; drawing and sharing the image.
quantity.ts, date.ts	Weighed vs countable servings and the stepper grid; date formatting.
voice.ts, install-prompt.ts	Recording and WAV conversion; the PWA install offer.
access.ts	Free vs premium as the database reports it; the plan line.
food/types.ts, index.ts, remote-provider.ts, local-provider.ts, provider-status.ts, indian-foods.ts	The provider contract and pure helpers (swap, totals); choosing remote or local; the Edge Function client; the offline parser; the online/offline observable; the curated table.
supabase/client.ts, config.ts, auth.tsx, supabase-kv.ts, consent.ts, account.ts, access.ts, programs.ts, admin.ts	Client and config; auth state and Google sign in; the cloud store and migration; consent; export and delete; access fetch; programs and follows; every admin read and write.
train/exercises.ts, workout.ts, store.ts, progress.ts, program.ts, adherence.ts, describe.ts, importer.ts	Library and matching; set maths and sessions; offline-first storage; progress and the one line; programs and positions; the coach summary; the workout text reader; imports.
admin/crm.ts, admin/format.ts	CRM urgency, views, sorting, funnel, CSV; counts, growth, stages, follow-ups, gift lengths.

Every file above with a matching `.test.ts` is covered by the 333 tests that pass on this version of the code.